GOVERNMENT ENGINEERING COLLEGE, BHARUCH

Student's Weekly Report of Internship

Student Name:	PATEL BINIT UMESHKUMAR
Enrollment Number:	230143107014
Week Duration:	February 26, 2026 - March 04, 2026 (Week 9)
Name Of Organization:	Microsoft Elevate Program (via Edunet Foundation & FICE Education)
External Guide Name:	Adarsh Gupta
External Guide Contact:	0124-4080107

Work done in last Week with description:

1. Week 9 Module: Azure Experiments - Blob Storage (Session 1)

Description:

Attended first hands-on "Azure Experiments" session on March 20, 2026, focused entirely on Azure Blob Storage practical implementation. This session provided deep dive into object storage operations, access tiers, security configurations, and Python SDK usage.

Key Topics Covered:

- Azure Blob Storage architecture: Account → Container → Blob hierarchy
- Storage Account creation and configuration (performance tiers, replication options)
- Access tiers: Hot (frequent access), Cool (30+ days), Archive (180+ days)
- Blob types: Block blobs (general files), Append blobs (logs), Page blobs (VHD)
- Container creation and naming conventions
- Public vs Private access levels for containers
- Shared Access Signatures (SAS) for time-limited access
- Access keys and connection strings management
- Lifecycle management policies for automatic tier transitions
- Blob versioning and soft delete for data protection
- Encryption at rest and in transit
- Azure Storage Explorer tool for GUI-based management
- Cost optimization strategies for Blob Storage

Hands-on Lab Exercises:

- Created Storage Account 'elevatelab2026' in Central India region
- Created three containers: 'documents', 'images', 'backups' with different access levels
- Uploaded files via Azure Portal (drag-drop interface)
- Downloaded blobs and verified data integrity
- Generated SAS token with 24-hour expiry and tested access
- Changed blob access tier from Hot to Cool and monitored cost impact
- Configured lifecycle policy to auto-move blobs to Cool tier after 30 days
- Installed Azure Storage Explorer and connected to storage account

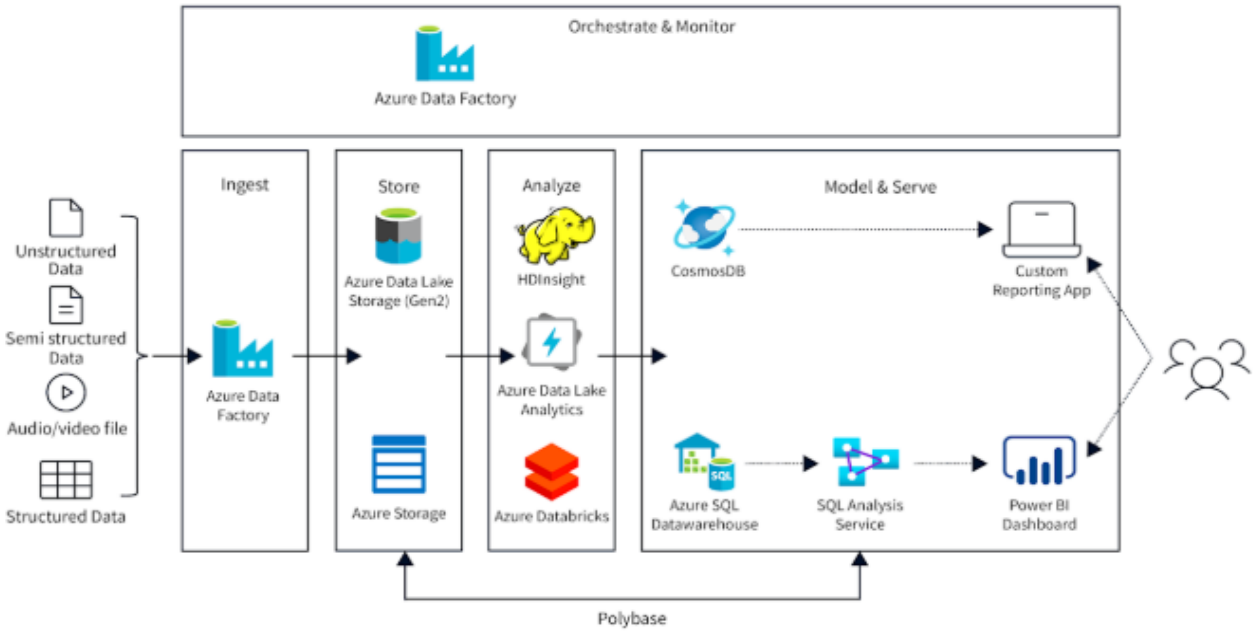

Figure 1: Azure Blob Storage hierarchy showing storage account, containers, and blobs used for object storage.

Python SDK Practice:

Installed azure-storage-blob library. Wrote Python script to upload multiple files to container. Implemented download function with progress tracking. Created list_blobs function to enumerate container contents. Practiced deleting blobs programmatically. Learned blob metadata management (custom key-value pairs). Implemented error handling for connection failures and authentication issues.

Assignment Completed:

Built Python application to backup local folder to Azure Blob Storage. Implemented incremental backup (upload only changed files). Added logging functionality to track uploaded files with timestamps. Created restore function to download blobs back to local system. Tested with 50+ sample files totaling 200MB. Submitted code with documentation and test results.

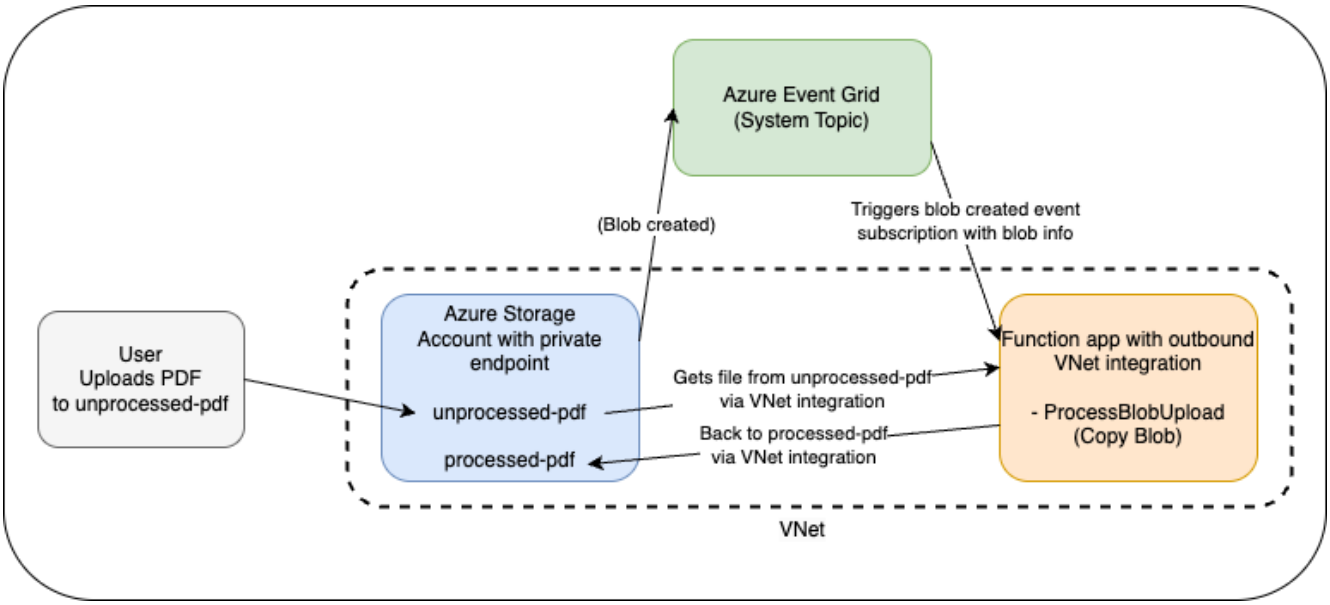

Figure 2: Event-driven processing pipeline where uploading a file to Blob Storage automatically triggers an Azure Function.

2. Week 9 Module: Azure Experiments - Azure Functions (Session 2)

Description:

Attended second "Azure Experiments" session on March 23, 2026, dedicated to serverless computing with Azure Functions. Learned to create, deploy, and manage event-driven functions with various trigger types and bindings.

Key Topics Covered:

- Serverless computing concepts and benefits (pay-per-execution, auto-scaling)
- Azure Functions architecture and execution model
- Function App creation and configuration
- Trigger types: HTTP, Timer, Blob, Queue, Event Grid, Service Bus
- HTTP triggers: GET, POST methods, request/response handling
- Timer triggers: Cron expressions for scheduled execution
- Blob triggers: Automatic execution on file upload
- Input and output bindings for connecting to other services
- Supported languages: Python, C#, JavaScript, Java, PowerShell
- Durable Functions for stateful workflows
- Application Insights integration for monitoring and logging
- Environment variables and application settings
- Consumption plan vs Premium plan pricing
- Deployment methods: Portal, VS Code, Azure CLI, GitHub Actions

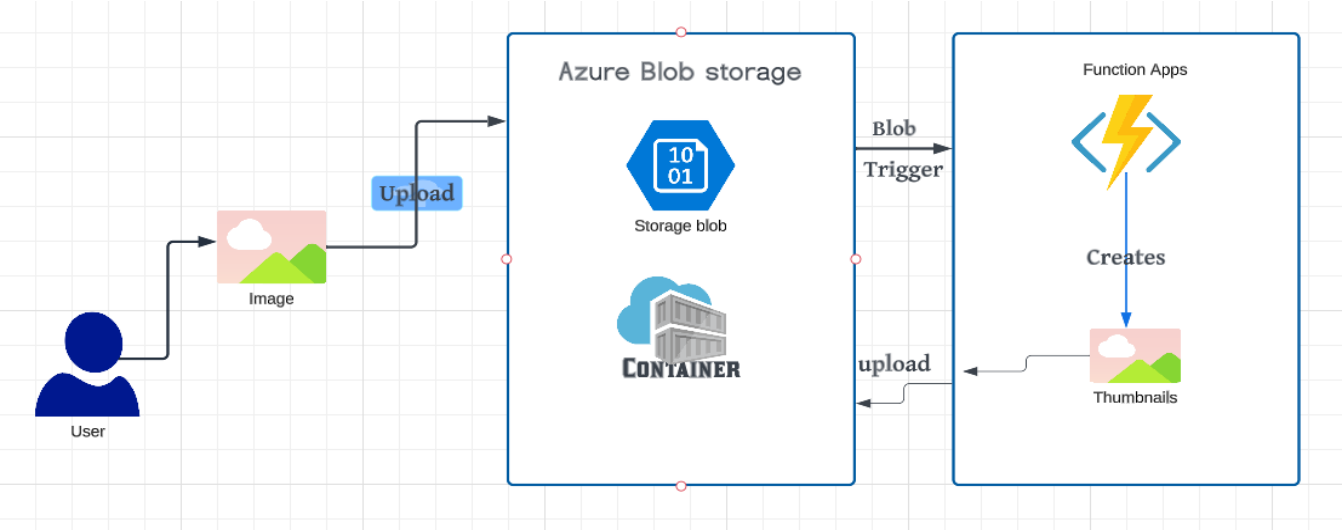

Figure 3: Serverless architecture using Azure Functions with triggers, bindings, and event-driven execution.

Hands-on Lab Exercises:

- Created Function App 'elevateFunctions2026' with Python 3.10 runtime
- Developed HTTP-triggered function to process JSON data (calculator API)
- Created Timer-triggered function to run daily at 9 AM (0 0 9 * * *)
- Built Blob-triggered function to process uploaded images
- Tested functions using Azure Portal test interface
- Configured output binding to write results to another Blob container
- Used Postman to test HTTP function endpoints
- Monitored function executions in Application Insights
- Analyzed logs and performance metrics

Python Function Development:

Developed HTTP function to accept student data (name, marks) and calculate grade. Implemented input validation and error handling. Created Timer function to send daily email report using SendGrid binding. Built Blob-triggered function to resize uploaded images using Pillow library. Configured function.json for triggers and bindings. Tested local development using Azure Functions Core Tools. Deployed functions to Azure using VS Code extension.

Assignment Completed:

Created serverless file processing pipeline: (1) User uploads file to Blob Storage, (2) Blob trigger activates function, (3) Function processes file (validate, transform), (4) Processed file written to output container, (5) HTTP function provides status API. Implemented error handling and retry logic. Created complete documentation with architecture diagram. Successfully processed 20 test files. Submitted code and demonstration video.

3. Week 9 Module: Azure Experiments - Cosmos DB (Session 3)

Description:

Attended third "Azure Experiments" session on March 27, 2026, covering Azure Cosmos DB - Microsoft's globally distributed, multi-model NoSQL database service. Learned document storage, querying, partitioning, and performance optimization.

Key Topics Covered:

- Azure Cosmos DB overview: globally distributed, multi-region writes
- API options: Core (SQL), MongoDB, Cassandra, Gremlin, Table
- Document-based NoSQL data model (JSON documents)
- Database and container hierarchy
- Partition key selection and importance for scalability
- Request Units (RUs): throughput measurement and pricing
- Consistency levels: Strong, Bounded staleness, Session, Consistent prefix, Eventual
- SQL query syntax for Cosmos DB (SELECT, WHERE, ORDER BY)
- Indexing policies: automatic indexing vs custom policies
- Change feed for real-time data processing
- Stored procedures, triggers, and user-defined functions (UDFs)
- Multi-region replication and failover configuration
- Backup and restore options
- Python SDK (azure-cosmos) for CRUD operations
- Cost optimization: serverless vs provisioned throughput

Hands-on Lab Exercises:

- Created Cosmos DB account 'elevatecosmosdb' with Core SQL API
- Created database 'StudentDB' with container 'Students'
- Selected partition key '/enrollmentNo' for efficient querying
- Provisioned 400 RU/s throughput for container
- Inserted sample documents using Data Explorer
- Executed SQL queries to filter and sort documents
- Updated documents using REPLACE operation
- Deleted documents using WHERE clause
- Analyzed query execution metrics and RU consumption
- Configured geo-replication to add Southeast Asia region

Python SDK Practice:

Installed azure-cosmos library. Connected to Cosmos DB using connection string and key. Implemented CRUD operations: create_item(), read_item(), query_items(), replace_item(), delete_item(). Built student management system with add, search, update, delete functions. Implemented pagination for large query results. Added error handling for duplicate keys and not found exceptions. Measured RU consumption for different operations. Practiced bulk insert for efficient data loading.

Assignment Completed:

Designed and implemented product catalog database for e-commerce application. Created container with partition key '/category'. Inserted 100+ product documents with fields: id, name, category, price, stock, description. Implemented search function by category, price range, keyword. Created aggregate queries: total products per category, average price. Built inventory management functions: update stock, low-stock alerts. Submitted Python application with complete CRUD functionality and documentation.

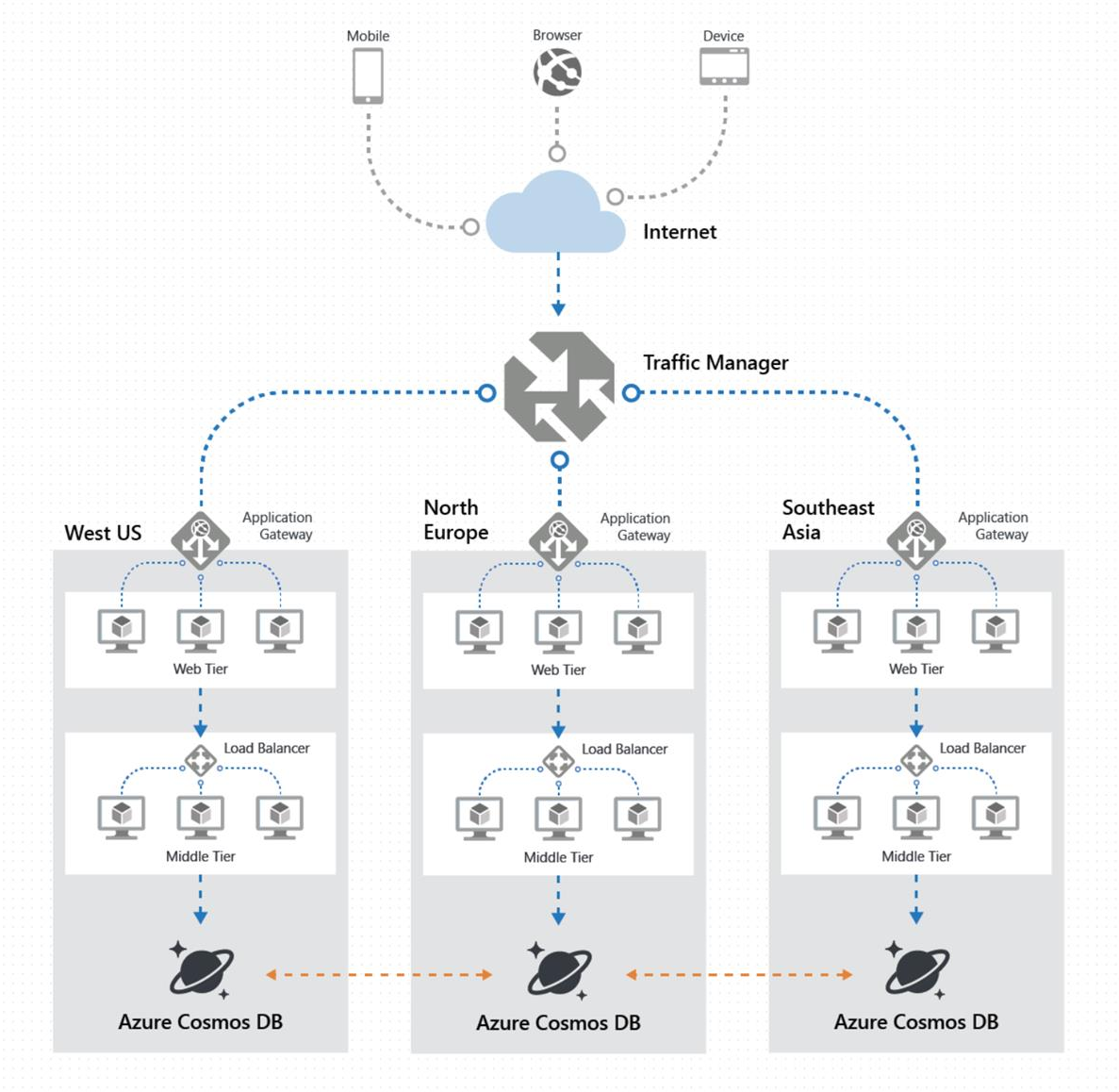

Figure 4: Azure Cosmos DB architecture showing distributed NoSQL database with containers, partitions, and JSON documents.

4. Azure CLI and PowerShell Automation

Description:

Practiced command-line management of Azure resources using Azure CLI and PowerShell. Learned automation scripts for resource deployment and management, which is essential for DevOps practices.

Skills Developed:

- Installed Azure CLI on Windows system
- Authenticated using 'az login' command
- Created resource group using CLI: az group create
- Listed all storage accounts: az storage account list
- Created storage container using CLI commands
- Uploaded blob using az storage blob upload
- Listed function apps and their configurations

- Wrote bash scripts for automated deployment
- Practiced PowerShell cmdlets for Azure (Az module)
- Automated resource cleanup using scripts

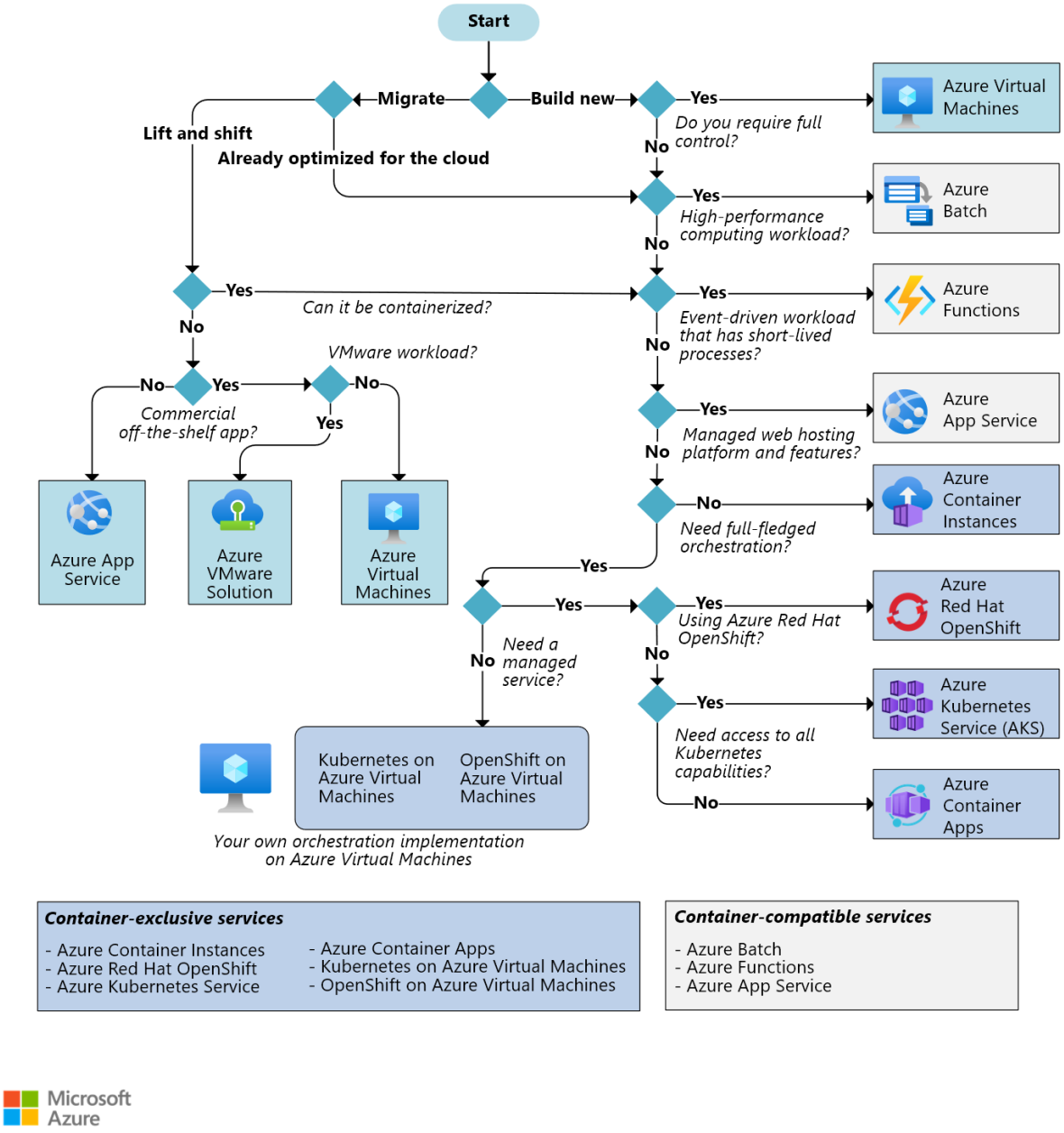

Figure 5: Azure CLI command-line interface used for automating resource deployment and management.

5. Azure Fundamentals Certification Preparation

Description:

Started preparing for Microsoft Azure Fundamentals (AZ-900) certification exam. This certification validates foundational knowledge of cloud concepts and Azure services, which will be valuable for career development.

Study Activities:

- Reviewed Microsoft Learn modules for AZ-900 exam
- Completed 'Cloud Concepts' learning path
- Studied Azure architecture and services
- Practiced sample questions from Microsoft practice exams
- Reviewed Azure pricing and support documentation
- Studied Azure security, privacy, and compliance concepts
- Created flashcards for key Azure services and terminology

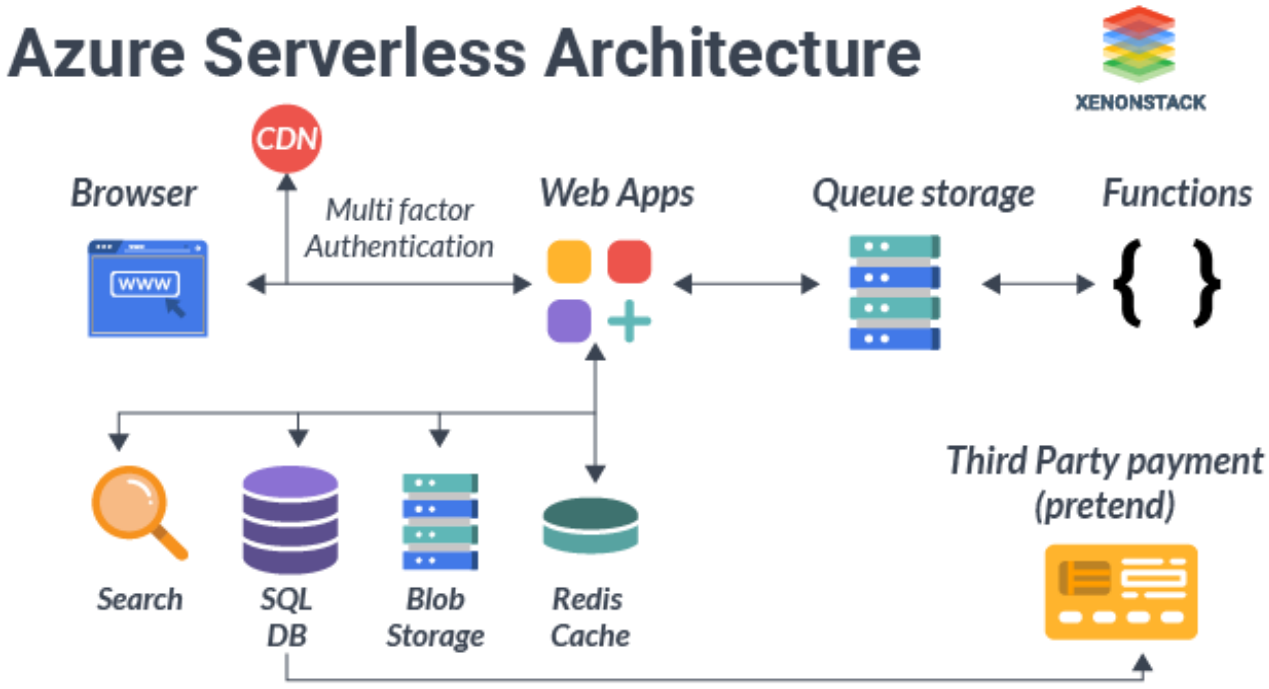

Figure 6: Proposed Azure architecture integrating Blob Storage for data, Azure Functions for processing, and Cosmos DB for metadata storage.

6. GTU Project - Architecture Planning and Documentation

Description:

Applied Week 9 Azure learning to map project requirements to appropriate Azure services. Created high-level architecture diagram showing how Blob Storage, Functions, and Cosmos DB will integrate in the surveillance system. This is planning work only - implementation will occur after completing remaining training modules.

Planning Work Completed:

- Created Azure architecture diagram for surveillance project
- Mapped video storage → Azure Blob Storage (Hot tier)
- Mapped detection processing → Azure Functions (Blob trigger)
- Mapped alert storage → Azure Cosmos DB (JSON documents)
- Estimated monthly Azure costs for project infrastructure
- Updated GitHub repository README with architecture diagram
- Documented Azure services selection rationale

Note:

Focus remains on Microsoft Elevate training. Actual Azure deployment and integration will begin in Week 10-11 after completing all cloud modules and certification preparation. Current priority is building strong Azure fundamentals through hands-on labs.

Plans for next week:

1. Complete Week 10 Microsoft Elevate Modules

- Attend remaining Azure modules and advanced topics
- Learn Azure DevOps and CI/CD pipelines
- Practice Azure Monitor and Application Insights
- Complete all pending Azure assignments

2. Continue AZ-900 Certification Preparation

- Complete remaining Microsoft Learn modules
- Take full-length practice exams
- Review weak areas based on practice test results
- Schedule certification exam for Week 11-12

3. Power BI Review and Practice

- Review Power BI concepts learned in Week 3-4
- Practice creating dashboards with sample datasets
- Learn Power BI integration with Azure services
- Prepare dashboard mockup for project visualization

4. Final Project Integration Planning

- Finalize project implementation timeline for Week 11-12
- Create detailed task breakdown for remaining work
- Prepare project presentation outline
- Review all datasets and tools needed

5. Internship Reflection and Documentation

- Compile learning portfolio from all 9 weeks
- Document key skills acquired in each module
- Update LinkedIn profile with new skills and certifications
- Prepare for final internship presentation

Total Working Hours: 36 hours

Signature of Student: _____

The above entries are correct, and the grading of work done by Trainee is:

Excellent	Very Good	Good	Fair	Below Average	Poor
☐	☐	☐	☐	☐	☐

Signature of External Guide with Company Seal: _____

Date: _____

Signature of Internal Guide: _____

Date: _____